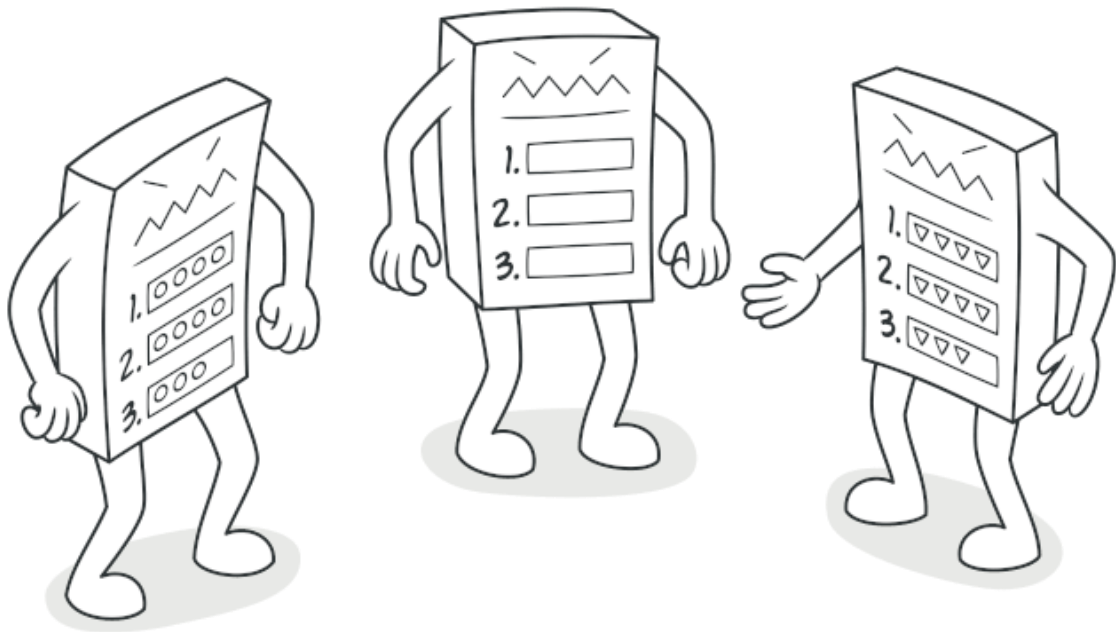


Template Method

También llamado: Método plantilla

Propósito

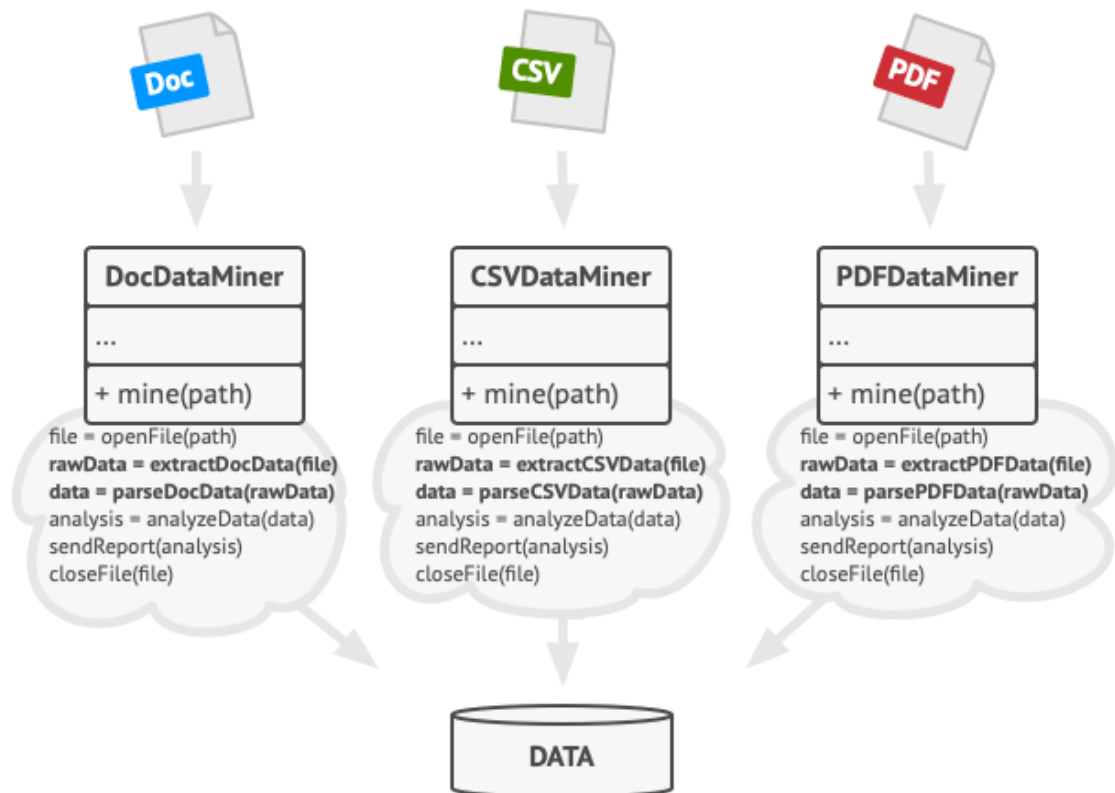
Template Method es un patrón de diseño de comportamiento que define el esqueleto de un algoritmo en la superclase pero permite que las subclasses sobrescriban pasos del algoritmo sin cambiar su estructura.



Problema

Imagina que estás creando una aplicación de minería de datos que analiza documentos corporativos. Los usuarios suben a la aplicación documentos en varios formatos (PDF, DOC, CSV) y ésta intenta extraer la información relevante de estos documentos en un formato uniforme.

La primera versión de la aplicación sólo funcionaba con archivos DOC. La siguiente versión podía soportar archivos CSV. Un mes después, le “enseñaste” a extraer datos de archivos PDF.



Las clases de minería de datos contenían mucho código duplicado. En cierto momento te das cuenta de que las tres clases tienen mucho código similar. Aunque el código para gestionar distintos formatos de datos es totalmente diferente en todas las clases, el código para procesar y analizar los datos es casi idéntico. ¿No sería genial deshacerse de la duplicación de código, dejando intacta la estructura del algoritmo?

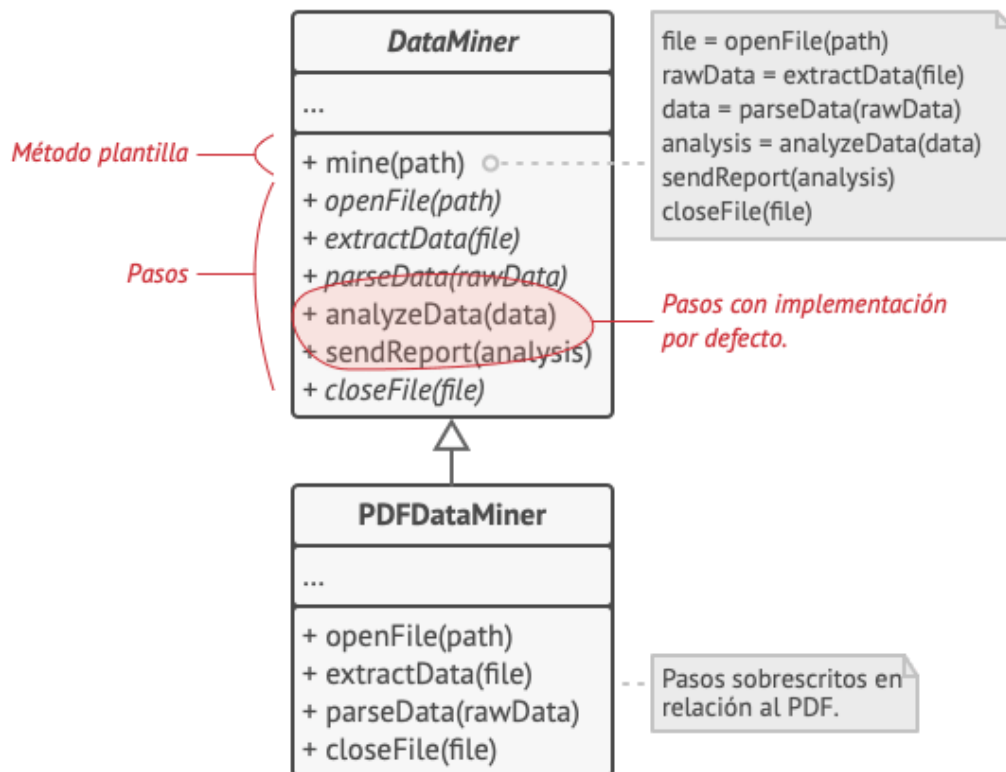
Hay otro problema relacionado con el código cliente que utiliza esas clases. Tiene muchos condicionales que eligen un curso de acción adecuado dependiendo de la clase del objeto de procesamiento. Si las tres clases de procesamiento tienen una interfaz común o una clase base, puedes eliminar los condicionales en el código cliente y utilizar el polimorfismo al invocar métodos en un objeto de procesamiento.

Solución

El patrón Template Method sugiere que dividas un algoritmo en una serie de pasos, conviertas estos pasos en métodos y coloques una serie de llamadas a esos métodos dentro de un único *método plantilla*. Los pasos pueden ser **abstractos**, o contar con una implementación por defecto. Para utilizar el algoritmo, el cliente debe aportar su propia subclase, implementar todos los pasos abstractos y sobrescribir algunos de los opcionales si es necesario (pero no el propio método plantilla).

Veamos cómo funciona en nuestra aplicación de minería de datos. Podemos crear una clase base para los tres algoritmos de análisis. Esta clase define un

método plantilla consistente en una serie de llamadas a varios pasos de procesamiento de documentos.



El método plantilla divide el algoritmo en pasos, permitiendo a las subclases sobrescribir estos pasos pero no el método en sí.

Al principio, podemos declarar todos los pasos como *abstractos*, forzando a las subclases a proporcionar sus propias implementaciones para estos métodos. En nuestro caso, las subclases ya cuentan con todas las implementaciones necesarias, por lo que lo único que tendremos que hacer es ajustar las firmas de los métodos para que coincidan con los métodos de la superclase.

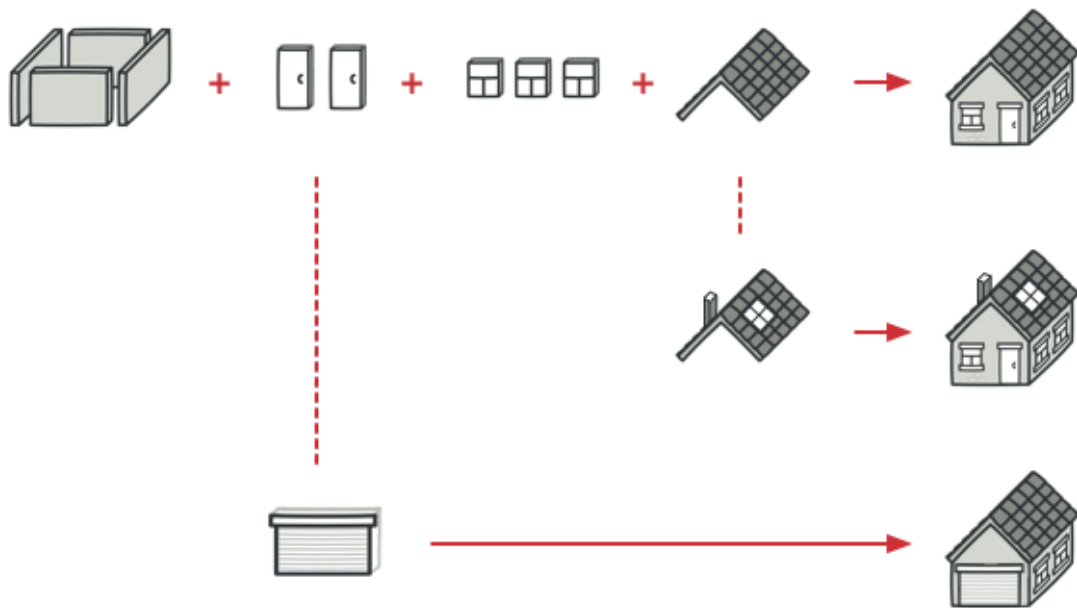
Ahora, veamos lo que podemos hacer para deshacernos del código duplicado. Parece que el código para abrir/cerrar archivos y extraer/analizar información es diferente para varios formatos de datos, por lo que no tiene sentido tocar estos métodos. No obstante, la implementación de otros pasos, como analizar los datos sin procesar y generar informes, es muy similar, por lo que puede meterse en la clase base, donde las subclases pueden compartir ese código.

Como puedes ver, tenemos dos tipos de pasos:

- Los *pasos abstractos* deben ser implementados por todas las subclases
- Los *pasos opcionales* ya tienen cierta implementación por defecto, pero aún así pueden sobrescribirse si es necesario

Hay otro tipo de pasos, llamados ganchos (*hooks*). Un gancho es un paso opcional con un cuerpo vacío. Un método plantilla funcionará aunque el gancho no se sobrescriba. Normalmente, los ganchos se colocan antes y después de pasos cruciales de los algoritmos, suministrando a las subclases puntos adicionales de extensión para un algoritmo.

Analogía en el mundo real

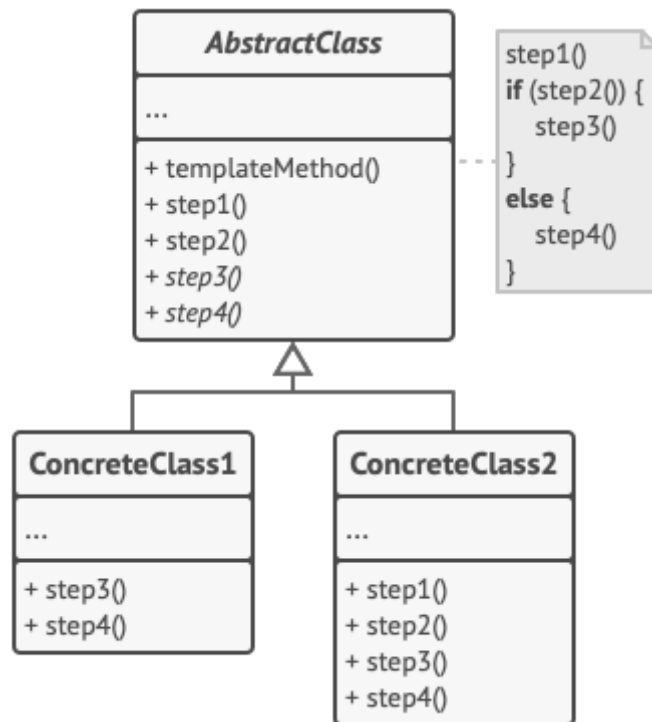


Un plan arquitectónico típico puede alterarse ligeramente para que encaje mejor con las necesidades del cliente.

El enfoque del método plantilla puede emplearse en la construcción de viviendas en masa. El plan arquitectónico para construir una casa estándar puede contener varios puntos de extensión que permitirán a un potencial propietario ajustar algunos detalles de la casa resultante.

Cada paso de la construcción, como colocar los cimientos, el armazón, construir las paredes, instalar las tuberías para el agua y el cableado para la electricidad, etc., puede cambiarse ligeramente para que la casa resultante sea un poco diferente de las demás.

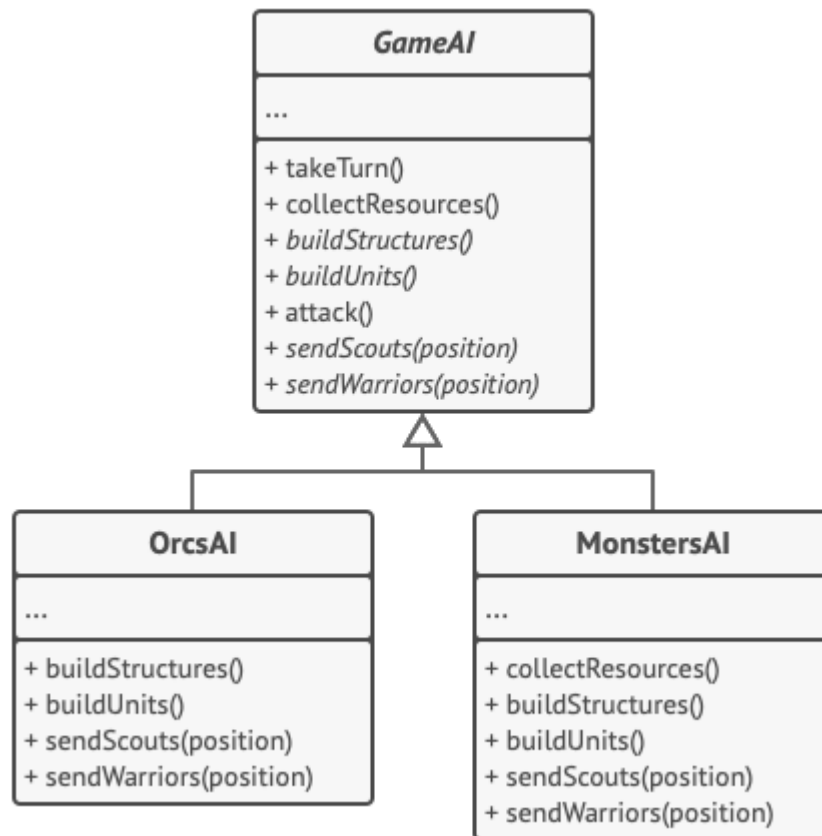
Estructura



1. La **Clase Abstracta** declara métodos que actúan como pasos de un algoritmo, así como el propio método plantilla que invoca estos métodos en un orden específico. Los pasos pueden declararse abstractos o contar con una implementación por defecto.
2. Las **Clases Concretas** pueden sobrescribir todos los pasos, pero no el propio método plantilla.

Pseudocódigo

En este ejemplo, el patrón **Template Method** proporciona un “esqueleto” para varias ramas de inteligencia artificial (IA) en un sencillo videojuego de estrategia.



Clases IA de un sencillo videojuego.

Todas las razas del juego tienen tipos de unidades y edificios casi iguales. Por lo tanto, puedes reutilizar la misma estructura IA para varias de ellas, a la vez que puedes sobrescribir algunos de los detalles. Con esta solución, puedes sobrescribir la IA de los orcos para que sean más agresivos, hacer que los humanos tengan una actitud más defensiva y hacer que los monstruos no puedan construir nada. Para añadir una nueva raza al juego habría que crear una nueva subclase IA y sobrescribir los métodos por defecto declarados en la clase IA base.

```

// La clase abstracta define un método plantilla que contiene un
// esqueleto de algún algoritmo compuesto por llamadas,
// normalmente a operaciones primitivas abstractas. Las
// subclases concretas implementan estas operaciones, pero dejan
// el propio método plantilla intacto.
class GameAI is
    // El método plantilla define el esqueleto de un algoritmo.
    method turn() is
        collectResources()
        buildStructures()
        buildUnits()
        attack()

    // Algunos de los pasos se pueden implementar directamente
    // en una clase base.
    method collectResources() is
        foreach (s in this.builtStructures) do
            s.collect()

    // Y algunos de ellos pueden definirse como abstractos.
  
```

```

abstract method buildStructures()
abstract method buildUnits()

// Una clase puede tener varios métodos plantilla.
method attack() is
    enemy = closestEnemy()
    if (enemy == null)
        sendScouts(map.center)
    else
        sendWarriors(enemy.position)

abstract method sendScouts(position)
abstract method sendWarriors(position)

// Las clases concretas tienen que implementar todas las
// operaciones abstractas de la clase base, pero no deben
// sobrescribir el propio método plantilla.
class OrcsAI extends GameAI is
    method buildStructures() is
        if (there are some resources) then
            // Construye granjas, después cuarteles y después
            // fortaleza.

    method buildUnits() is
        if (there are plenty of resources) then
            if (there are no scouts)
                // Crea peón y añádelo al grupo de exploradores.
            else
                // Crea soldado, añádelo al grupo de guerreros.

// ...

    method sendScouts(position) is
        if (scouts.length > 0) then
            // Envía exploradores a posición.

    method sendWarriors(position) is
        if (warriors.length > 5) then
            // Envía guerreros a posición.

// Las subclases también pueden sobrescribir algunas operaciones
// con una implementación por defecto.
class MonstersAI extends GameAI is
    method collectResources() is
        // Los monstruos no recopilan recursos.

    method buildStructures() is
        // Los monstruos no construyen estructuras.

    method buildUnits() is
        // Los monstruos no construyen unidades.

```

Aplicabilidad

Utiliza el patrón Template Method cuando quieras permitir a tus clientes que extiendan únicamente pasos particulares de un algoritmo, pero no todo el algoritmo o su estructura.

El patrón Template Method te permite convertir un algoritmo monolítico en una serie de pasos individuales que se pueden extender fácilmente con subclases, manteniendo intacta la estructura definida en una superclase.

Utiliza el patrón cuando tengas muchas clases que contengan algoritmos casi idénticos, pero con algunas diferencias mínimas. Como resultado, puede que tengas que modificar todas las clases cuando el algoritmo cambie.

Cuando conviertes un algoritmo así en un método plantilla, también puedes elevar los pasos con implementaciones similares a una superclase, eliminando la duplicación del código. El código que varía entre subclases puede permanecer en las subclases.

Cómo implementarlo

1. Analiza el algoritmo objetivo para ver si puedes dividirlo en pasos. Considera qué pasos son comunes a todas las subclases y cuáles siempre serán únicos.
2. Crea la clase base abstracta y declara el método plantilla y un grupo de métodos abstractos que representen los pasos del algoritmo. Perfila la estructura del algoritmo en el método plantilla ejecutando los pasos correspondientes. Considera declarar el método plantilla como `final` para evitar que las subclases lo sobrescriban.
3. No hay problema en que todos los pasos acaben siendo abstractos. Sin embargo, a algunos pasos les vendría bien tener una implementación por defecto. Las subclases no tienen que implementar esos métodos.
4. Piensa en añadir ganchos entre los pasos cruciales del algoritmo.
5. Para cada variación del algoritmo, crea una nueva subclase concreta. Ésta *debe* implementar todos los pasos abstractos, pero también *puede* sobrescribir algunos de los opcionales.

Pros y contras

- Puedes permitir a los clientes que sobrescriban tan solo ciertas partes de un algoritmo grande, para que les afecten menos los cambios que tienen lugar en otras partes del algoritmo.
- Puedes colocar el código duplicado dentro de una superclase.
- Algunos clientes pueden verse limitados por el esqueleto proporcionado de un algoritmo.
- Puede que violes el *principio de sustitución de Liskov* suprimiendo una implementación por defecto de un paso a través de una subclase.
- Los métodos plantilla tienden a ser más difíciles de mantener cuantos más pasos tengan.

Relaciones con otros patrones

- **Factory Method** es una especialización del **Template Method**. Al mismo tiempo, un *Factory Method* puede servir como paso de un gran *Template Method*.

- **Template Method** se basa en la herencia: te permite alterar partes de un algoritmo extendiendo esas partes en subclases. **Strategy** se basa en la composición: puedes alterar partes del comportamiento del objeto suministrándole distintas estrategias que se correspondan con ese comportamiento. *Template Method* trabaja al nivel de la clase, por lo que es estático. *Strategy* trabaja al nivel del objeto, permitiéndote cambiar los comportamientos durante el tiempo de ejecución.